

中心主题

单例模式

原型模式

分支主题 3

工厂方法模式

建造者模式

抽象工厂模式

代理模式

组合模式

享元模式

外观模式 Facade Pattern

适配器模式

装饰器模式

- 组合模式
  - 优点
    - 在树状结构中，你可以统一对待组合对象（分支）和单个对象（叶节点）。
    - 使用这种设计模式来实现“部分-整体”层次结构是非常常见的。
    - 可以轻松地向现有架构添加新组件，或者从架构中删除现有组件。
  - 开销增大
    - 可以轻松添加新组件，但这方面的支持可能会在未来导致维护开销。有时，你想处理一个具有特殊组件的组合对象。这种约束可能会导致额外的开发成本，因为你可能需要实现动态检查机制来支持这一概念。
  - 缺点
    - 难以改变节点顺序
      - 如果你想保持子节点的顺序（例如，如果解析树被表示为组件），你可能需要付出额外的努力。
    - 不好删除不可变对象
      - 如果你处理的是不可变对象，你不能简单地删除它们。

- 享元模式
  - 优点：
    - 什么是“重量级对象”？士兵的 3D 模型文件、高清纹理贴图、骨骼动画数据。这些数据很大，假设一个士兵的模型数据要占 10MB 内存。
    - 如果不优化：创建 10,000 个士兵，每个士兵对象里都加载一份 10MB 的模型数据。内存消耗 = 10,000 × 10MB = 100GB。内存直接爆炸。
    - 优化后（享元模式）：无论有多少个士兵，他们其实长得都一样（或者只有红蓝两种样子）。我们在内存里只加载 1 份士兵的模型数据（10MB）
  - 优缺点
    - 你可以减少那些可以被相同控制的重量级对象的内存消耗。
    - 你可以减少系统中“完整但相似对象”的总数。
    - 你可以提供一种集中机制来控制许多“虚拟”对象的状态。
  - 概念
    - 因为这 5,000 个虚拟士兵都共享同一个红队士兵模板。你只需要修改这 1 个模板对象的颜色。
  - 缺点：
    - 在这种模式中，你需要花时间来配置这些享元。配置时间可能会影响应用程序的整体性能。
    - 为了创建享元，你需要从现有对象中提取一个通用的模板类。这一层额外的编程可能会很棘手，有时难以调试和维护。
    - 你会发现一个类的逻辑实例无法表现得与其他实例不同（注：指共享的内部状态部分）。
    - 享元模式通常与单例工厂实现结合使用，为了守护单一性，需要额外的成本（例如，你可能会选择同步方法或双重检查锁定，但它们每一个都是昂贵的操作）。

- 外观模式 Facade Pattern
  - 概念
    - 使用共享技术来高效地支持大量细粒度对象。当需要大量相似对象，而这些对象的大部分状态可以共享（内在状态）时，该模式通过重用对象来最小化内存使用。
  - 例子：公司可以创建一个通用的名片模板，包含公司标志、地址等内在状态（共享数据）。然后，为每位员工添加独特的外在状态（姓名、电话号码等），这些数据由客户端（员工信息）维护并传递给模板。
  - 优缺点
    - 为子系统的一组复杂接口提供一个统一且简化的接口。它定义了一个高层接口，使子系统更容易被客户端使用。
    - 通过提供一个简化的统一接口，降低客户端与复杂子系统之间的耦合，提升易用性和可维护性。
    - 缺点：增加了系统复杂性，且当子系统变更时需同步维护外观层，同时可能掩盖已有开发者对子系统的直接高效使用。

- 适配器模式
  - 分类
    - 类适配器：通过子类化进行适配。它们是多继承的倡导者。但在 Java 不支持通过类实现多继承，你需要继承一个现有的类并实现所需的接口。
    - 对象适配器
      - (1) 你的类需要实现目标接口 RectInterface。如果目标是抽象类，你需要继承它。
      - (2) 在构造函数中提及你正在适配的类，并在实例变量中存储对该类的引用。
      - (3) 重写接口方法以委托你所适配的类的相应方法。
  - 优缺点
    - 适配器模式的主要缺点是增加了系统的复杂性。引入适配器类意味着添加了额外的代码和类，使系统结构变得更加复杂
    - 将一个类的接口转换成客户端期望的另一个接口。适配器模式使原本由于接口不兼容而不能一起工作的类可以协同工作。

- 装饰器模式
  - 优缺点
    - 允许在不改变原有代码和对象结构的前提下，动态地、逐层地为对象添加新功能，支持灵活扩展和渐进式开发。
    - 太多的装饰器将很难维护和调试
  - 为什么直接继承而是要使用装饰器模式？
    - 用装饰器
      - 可以像搭积木一样，按需叠加“加楼层”“粉刷”等包装器，任意组合
      - 装饰器模式支持运行时动态、灵活、可组合地添加或移除职责；
    - 继承
      - 每种功能组合都需要一个新子类（如“加楼层+粉刷”“仅粉刷”“仅加楼层”），导致类数量激增，且无法灵活应对新需求（比如客户只想粉刷但不加楼层）。
      - 而继承是静态的、僵化的，容易导致类爆炸、代码重复和无法满足任意功能组合

- 例子
  - 假设您有一个非常稳定的应用程序。将来，您可能想要对其进行一些小的修改来更新它。所以，您从原始应用程序的副本开始，进行修改并进一步分析。当然，为了节省您的时间和金钱，您肯定不想从头开始。
  - 真实世界
    - 假设我们有一份珍贵文件的原件。我们需要对其进行一些修改以查看修改的效果。在这种情况下，我们可以复印一份原件并进行编辑修改。
    - 再考虑另一个例子。假设一群人决定为他们的朋友罗恩庆祝生日。他们去了一家面包店买了一个蛋糕。为了使其特别，他们要求卖家在蛋糕上写上“罗恩生日快乐”。
  - 浅拷贝：
    - 先创建一个新的对象，然后将原始对象中的各种字段值复制到新对象中。
    - 更快且成本更低。如果你的目标对象只有基本字段，那么它总是更好的选择。
  - 额外
    - 深拷贝，
      - 新对象与原始对象完全分离。任何更改都不反映在另一个对象上。
      - 成本高且速度慢。但如果你目标对象包含许多字段，这些字段引用了其他对象，那么它是有用的。

- 工厂方法模式
  - 优缺点
    - 解耦创建什么与怎么创建，易于扩展新类而无需修改现有代码
      - 比如说创建一个类很麻烦，那么相比直接new 一个会好很多，创建某个对象贼麻烦的时候，工厂方式好用
    - 为每个新产品创建对应的工厂子类，导致类爆炸和维护成本上升
    - 在工厂模式中，创建对象时不会对客户端暴露创建逻辑，并且是通过一个共同的接口来创建新的对象。
  - 与简单工厂模式区别
    - 工厂方法模式的关键在于它让一个类将实例化操作推迟到了子类中进行
      - 旨在解决这类代码难以维护的问题，通过继承和多态来处理对象创建，避免了修改原有代码的需要。
    - 简单工厂往往依赖于集中的条件判断逻辑

- 建造者模式
  - 优缺点
    - 优点
      - 能够将复杂对象的构建与表示分离，并使相同的构建过程生成不同的表示
    - 缺点
      - 会增加系统的复杂性和类的数量
  - 现实例子
    - 要完成一台电脑的订单，会根据客户的偏好组装不同的部件（例如，一位客户可以选择 500GB 的硬盘搭配英特尔处理器，而另一位客户则可以选择 250GB 的硬盘搭配 AMD 处理器）。
    - 再举个例子。假设您打算和一家旅游公司出游，该公司为同一条线路提供了多种套餐（比如，他们可以提供特别安排、不同类型的观光车辆等）。您可以根据自己的预算来选择套餐。
  - 计算机
    - 当我们需要将一种文本格式转换为另一种文本格式（例如，从 RTF 格式转换为 ASCII 文本格式）时，就可以使用构建者模式。
    - 构建过程是固定的（Director 的角色）：
      - 无论你要转换成什么格式，读取 RTF 文件的过程是一样的。你需要一个“阅读器”（扮演 Director 指挥者），负责读取 RTF 文档，识别出什么是“文本段落”、什么是“加粗指令”、什么是“换行符”。这个解析文档的算法是稳定且复杂的。
    - 最终表示是变化的（Builder 的角色）：虽然读取过程一样，但对读到的内容的处理方式不同
      - 如果目标是 ASCII（纯文本）：当读到“加粗”指令时，ASCII 建造者会选择忽略它，只保留文字。
      - 如果目标是 HTML：当读到“加粗”指令时，HTML 建造者会将其转换为 <b> 标签。
      - 如果目标是 PDF：PDF 建造者会将其转换为相应的二进制绘制指令。

- 抽象工厂模式
  - 三种工厂区别
    - 简单工厂模式
      - 由一个具体工厂类创建哪种动物，客户端不关心类型选择
      - 使用一个工厂对象用来生产同一等级结构中的任意产品
    - 工厂方法模式
      - 客户端先选择具体的工厂子类，每个工厂只创建一种固定类型的动物，将“创建什么”的决策权交给客户端
      - 使用多个工厂对象用来生产同一等级结构中对应的固定产品
    - 抽象工厂模式
      - 客户端选择一个能创建一族相关对象的工厂，该工厂可同时创建多个相关产品（如 WildDog + WildTiger），强调产品族的一致性。

- 代理模式
  - 优缺点
    - 优点
      - ①访问控制：代理可控制对真实对象的访问权限，提供额外的安全防护或验证机制。例如在保护性代理中，可限制特定用户访问敏感数据。
      - ②延迟初始化：支持延迟初始化机制，即真实对象仅在需要时创建，从而节省资源。
      - ③远程访问：代理可代表不同地址空间中的对象，例如分布式系统中位于远程服务器的真实对象。
      - ④日志记录与审计：代理可在转发请求至真实对象前进行日志记录，便于审计和监控。
      - ⑤性能优化：在缓存代理中，可将高频请求结果缓存以避免重复执行高成本计算。
  - 三种
    - 远程代理
      - 为位于不同地址空间（如远程服务器）的对象提供一个本地的代表。
    - 虚拟代理
      - 用于通过延迟加载（Lazy Loading）来节省系统资源，仅在真正需要时才创建高开销对象。
    - 保护代理
      - 用于控制对原始对象的访问权限，决定谁可以调用，谁不可以调用。
  - 代理模式和装饰器模式的区别？
    - 代理模式的目的是控制对对象的访问（如权限检查、延迟加载、保护等），通常与原对象实现相同接口，对外“透明”地代替真实对象。
    - 装饰器模式的目的是动态地给对象添加新职责或功能，通常通过组合并扩展行为，可能使用相同或扩展后的接口。